Why use a key in kafka producer?

In Kafka, a **key** is an optional field you can attach to each produced message. It's used primarily for **partitioning and ordering guarantees**, but it also enables other important behaviors. Here's why you'd use one:

1. Ordering Guarantees (Most Important Reason)

Kafka only guarantees message order **within a single partition**, not across a topic. Messages with the **same key always go to the same partition** (via the default partitioner: `hash(key) % numPartitions`).

So if you need events for a given entity processed in order, key by that entity:

```java
// All events for user 123 land in the same partition → processed in order
producer.send(new ProducerRecord<>("user-events", "user-123", event));
```

Without a key, messages are round-robin distributed and ordering is lost.

2. Co-location of Related Data

Because same-key messages share a partition, consumers can process all events for one entity on a single consumer instance — avoiding cross-instance coordination and race conditions. This is the foundation of patterns like:

- **Event sourcing** (per-aggregate ordering)
- **CDC / outbox** (per-row ordering)
- **Per-customer / per-device state machines**

3. Compaction (Log Compaction)

If a topic is configured with `cleanup.policy=compact`, Kafka retains the **latest value per key** and garbage-collects older ones. This turns a topic into a changelog / key-value store. Requires keys to be meaningful (e.g., `userId → latest profile`).

4. Idempotent / Deduplicating Producers

The idempotent producer and transactions rely on keys as part of the message identity for deduplication semantics across retries.

5. Predictable Partition Assignment / Locality

Keys let you control which partition data lands on, which can help with:

- **Co-partitioning** for stream joins (Kafka Streams requires co-partitioned topics to join)
- Balancing load intentionally (avoiding hot partitions by choosing good keys)

Trade-offs & Pitfalls

- **Hot partitions:** A skewed key distribution (e.g., one giant tenant) sends all its traffic to one partition → bottleneck. Use composite/salted keys if needed.
- **No key ≠ random forever:** With `null` keys, the default partitioner uses a "sticky" strategy (batching to one partition, then rotating) for efficiency — but you lose per-entity ordering.
- **Changing partition count re-hashes keys:** Adding partitions changes `hash(key) % N`, so the same key may move partitions → breaks ordering/compaction assumptions mid-stream. Plan partition counts carefully (or use a custom partitioner/murmur2 with care).

Rule of Thumb

> **Use a key whenever message ordering or data locality matters for a given entity** (user, order, device, account). Omit the key when messages are independent and you just want throughput and even distribution.

> So when I read from a topic, do I read from a partition of it and not from the other partitions

Yes — that's exactly right, and it's a key mental model for Kafka. Let me break it down precisely.

Consumers read from specific partitions, not "the topic"

A **consumer never subscribes to a topic as a whole**. It subscribes to a topic, but the actual data flow is always **per partition**. The mechanics:

1. A consumer (or consumer group) is assigned a set of **partitions** — not the topic.
2. A `KafkaConsumer` maintains a **separate offset for each partition it's assigned** (`position` per `TopicPartition`).
3. Each `poll()` fetches from **each assigned partition independently**.

So when you call:

```
java

consumer.subscribe(List.of("orders"));
while (true) {
    ConsumerRecords<String, Order> records = consumer.poll(Duration.ofMillis(100));
    // records are merged from all partitions assigned to THIS consumer
}
```

...the returned `ConsumerRecords` is a **merged view** across the partitions *this consumer owns* — but under the hood it's separate fetches, and ordering is only guaranteed *within each partition*.

What a single consumer actually sees

A **single consumer instance** reads only the partitions **assigned to it**. It does *not* read the other partitions. Those are read by **other consumers in the same consumer group**.

```
text

Topic "orders" (3 partitions)
┌──────────────┐
| P0           |──► Consumer A
| P1           |──► Consumer A
| P2           |──► Consumer B
└──────────────┘
```

- Consumer A reads P0 and P1 only.
- Consumer B reads P2 only.

- Neither reads the other's partitions.

Who reads all partitions?

To read **all** partitions of a topic, you need **one consumer per partition, all in the same consumer group** (or a single consumer if it's the only one and gets everything assigned).

- **1 consumer, 3 partitions** → that one consumer is assigned all 3 (reads everything).
- **3 consumers, 3 partitions** → each gets one partition.
- **4 consumers, 3 partitions** → one consumer sits **idle** (no partition to assign). Extra consumers beyond partition count are wasted.

The group coordinator handles assignment via the partition assignor strategy (range, round-robin, sticky, cooperative-sticky).

Important consequences

1. **Max parallelism = number of partitions.** You can't have more *active* consumers in a group than partitions. This is why partition count caps consumer throughput.
2. **Ordering is per partition.** Even a single consumer processing P0 and P1 has **no cross-partition order** between them — P0's messages and P1's messages interleave arbitrarily. This is the whole reason keys matter: same key → same partition → you *do* get order for that key.
3. **Offsets are per partition.** `commitSync` stores an offset for each `TopicPartition`. Rebalance moves partition ownership, and the new owner resumes from the committed offset of *that partition* — not a single topic-wide offset.
4. **Manual assignment skips groups.** You can bypass the group machinery and pin a consumer to specific partitions:

```
java

consumer.assign(List.of(new TopicPartition("orders", 0)));
```

Now it reads *only* P0, no rebalancing. (Used for things like replay tools or partition-aware workers.)

Quick mental summary

Question	Answer
Does a consumer read "a topic"?	No — it reads **specific partitions** assigned to it.
Does one consumer read all partitions?	Only if it's the sole member and gets all assigned.
How do all partitions get read?	Across the **consumer group** — one partition per consumer (at most).
Is order guaranteed?	Only **within a partition**; never across partitions.
Can two consumers in a group read the same partition?	No — a partition is owned by exactly one consumer at a time.

So your statement is correct: **you read from a partition (or a set of them), never "the topic" as an indivisible unit** — the topic is just the logical grouping of its partitions.

So is one partition per consumer?

Short answer: **No — it's not strictly one partition per consumer.** The relationship is:

- **One partition can be owned by only one consumer** (within a group) at a time. ✅ (This part is fixed.)
- **One consumer can own multiple partitions.** ✅ (This happens often.)

So the "one-to-one" idea only holds in one specific case. Let me lay out the actual rule.

The real constraint

> Within a consumer group, **each partition is assigned to exactly one consumer**, but **one consumer may be assigned many partitions**.

That's it. The mapping is **many partitions → one consumer**, never one partition → many consumers (within a group).

The three regimes (N partitions, C consumers in a group)

C < N → consumers own multiple partitions

```
text

N=4 partitions, C=2 consumers
P0 ─┐
P1 ─┴─► Consumer A   (owns 2)
P2 ─┐
P3 ─┴─► Consumer B   (owns 2)
```

This is the **normal, healthy** case. Consumers are multi-partition workers.

C = N → one partition per consumer

```
text

N=3, C=3
P0 ──► A
P1 ──► B
P2 ──► C
```

This is the **only** case where "one partition per consumer" is literally true — and it's just the sweet spot for max parallelism, not a rule.

C > N → some consumers idle

```
text

N=2, C=3
P0 ──► A
P1 ──► B
     C  (idle — nothing to assign)
```

Extra consumers get **no partitions** and sit idle. This is why people say "don't run more consumers than partitions" — but again, it's about efficiency, not correctness.

Why it's many-to-one, not one-to-one

- **A partition is a unit of ordering + a unit of offset.** It must have a single owner so offsets and ordering aren't fought over → one partition, one consumer.
- **A consumer is just a polling loop.** Nothing stops it from fetching from 10 partitions in the same `poll()` call — it keeps a separate fetch position per partition internally. Assigning multiple partitions to one consumer is free and normal.

Practical implication

If you have 6 partitions and 2 consumers, **Consumer A reads 3 partitions and Consumer B reads 3 partitions** — both are reading "the topic," just different slices. To scale throughput, you add consumers up to the partition count; beyond that, extra consumers are dead weight (unless partitions are skewed and you want finer rebalancing granularity).

One nuance: cooperative vs eager rebalancing

During a **rebalance**, partitions move between consumers. With **eager** rebalancing (default `RangeAssignor` / `RoundRobinAssignor` behavior), all consumers revoke everything, then reassign — so momentarily a consumer may have 0 partitions, then several. With **cooperative-sticky** (`CooperativeStickyAssignor`), only the partitions that need to move are revoked — consumers keep the rest. Either way, the invariant "one partition → one consumer" holds throughout; the *number per consumer* just changes.

Summary table

Situation	Partition→Consumer mapping
Fewer consumers than partitions	1 consumer owns **many** partitions
Consumers == partitions	1-to-1 (the special case)
More consumers than partitions	Some consumers own **0** partitions
Across the whole group	Every partition owned by exactly **one** consumer

So: **a partition has exactly one consumer, but a consumer can have many partitions.** "One partition per consumer" is only the C = N case, not the general rule.

> So if i send an event to a partition, only one kafka consumer will receive it?

Correct — **yes, in the normal consumer-group model, a message in a partition is delivered to exactly one consumer in the group.** That's the core guarantee. But there are important nuances, so let me be precise about *what* "one consumer receives it" means.

The guarantee (within a consumer group)

A partition is owned by **exactly one consumer in a group at a time**, and that consumer reads every message in it. So for a given group:

> Each record in a partition is processed by **exactly one consumer instance** in that group.

This is what makes consumer groups a **scaling/load-balancing** mechanism — the work is divided, not duplicated.

```
text

Topic, P0 with messages m1,m2,m3
Group "billing":  A owns P0  → A gets m1,m2,m3
Group "analytics": B owns P0 → B gets m1,m2,m3
```

- Within **group "billing"**, only A gets them.
- But a **different group** ("analytics") independently gets all of them too.

The critical caveat: it's per *consumer group*, not globally

"Only one consumer receives it" is **only true within a single consumer group**. Different consumer groups each get their **own full copy** of the stream (each tracks its own offsets).

So if you have 3 different services each with their own group, **all 3 receive every message**. This is by design — it's how Kafka does pub/sub + queueing at the same time:

- **Same group** → queue semantics (load-balanced, one consumer per message)
- **Different groups** → pub/sub semantics (every group gets everything)

- **Different groups** → pub/sub semantics (every group gets everything)

Also: not "the partition" — the *record*

You send a record **to a topic**, and the producer's partitioner decides **which partition** it lands in (key hash or sticky/round-robin for null keys). Once it's in partition P, then the "one consumer per group owns P" rule applies. You don't address a partition directly unless you pass an explicit partition to `ProducerRecord`.

Edge cases where "exactly one" can bend

1. **Rebalance in flight.** Partitions move between consumers. With **eager** rebalancing there's a revoke/reassign window; a message committed-but-not-processed during the handoff can be **reprocessed** by the new owner (at-least-once). So "one consumer receives it" holds per *stable assignment*, but during rebalances you can get **duplicate delivery** to a second consumer.
2. **At-least-once / retries.** If a consumer processes but crashes before committing its offset, the next owner re-reads from the last committed offset → **the same message is delivered again**. Still "one at a time," but not "exactly once" unless you use transactions/idempotence and commit carefully.
3. **Standalone consumers / manual assignment.** A consumer using `assign()` (not `subscribe()`) isn't in the group balancing — you could point **two independent consumers at the same partition** and both read it. Kafka won't stop you; you just get duplicate processing. "One consumer per partition" is a *group* guarantee, not a broker-enforced global lock.
4. **Same group, two consumers, manual assign.** If you bypass the group coordinator, the "exactly one owner" invariant is **your** responsibility, not Kafka's.
5. **MirrorMaker / replication tools** read partitions directly and are outside your app's group — they "receive" the same messages too.

Precise statement

> For a **single consumer group**, each record in a partition is delivered to **exactly one consumer instance** of that group (at a given stable assignment), and that consumer reads the partition's records in order. **Other groups** each receive their own copy. During rebalances or after crashes, **at-least-once** redelivery means the *same* record can be handed to a *different* consumer instance over time.

> can be handled by a different consumer instance over time.

So your sentence is right **if you add "within a consumer group, and modulo redelivery."** It is *not* true that only one consumer *system-wide* ever sees the message — that's the whole point of multiple consumer groups.

> Okay, in my case, I want a topic that broadcasts to all of the subscribers

Then you want **one consumer group per subscriber** — that's exactly the built-in broadcast mechanism in Kafka. Let me spell out the model and the gotchas.

The core idea

Kafka's pub/sub vs. queueing is decided **entirely by how you set** `group.id` :

Goal	How	Semantics
Broadcast to all subscribers	Each subscriber uses a **distinct** `group.id`	Every group gets **every message**
Load-balance work	Consumers **share one** `group.id`	Each message goes to **one** consumer in the group

So for broadcast: **subscriber A → group** `"svc-a"` **, subscriber B → group** `"svc-b"` **, subscriber C → group** `"svc-c"` **.** Each gets the full stream, independently, with its own offsets.

```
text

Topic "events" (P0, P1)
            ┌─► Group "svc-a" → Consumer A  (gets everything)
broker ─────┼─► Group "svc-b" → Consumer B  (gets everything)
            └─► Group "svc-c" → Consumer C  (gets everything)
```

Each subscriber can still **scale horizontally within its own group** — e.g. `svc-a` runs 3 consumer instances all with `group.id="svc-a"` ; they split the partitions among themselves, but the *group as a whole* still sees every message. Best of both worlds.

Don't do this

- **Don't use `assign()` for broadcast.** Manual assignment bypasses groups, but then you manage offsets and partition ownership yourself, and you lose the group's offset tracking / rebalancing. Use distinct `group.id` s with `subscribe()` instead.
- **Don't give two logically-different subscribers the same `group.id`** — they'll steal partitions from each other and each get only a *slice* of the stream. That's the #1 broadcast bug.

Trade-offs / gotchas to plan for

1. **Each group = full copy of the data.** N subscribers × M bytes = N×M bytes read off the brokers and stored as N sets of offsets. Broadcast scales *read* load linearly with subscribers. Fine for a handful; watch broker/network egress if you have many.
2. **Each group has its own lag and offset lifecycle.** A slow subscriber doesn't block others (good), but it can fall far behind. Retention (`retention.ms` / `retention.bytes`) is **per partition, not per group** — if a subscriber is down longer than retention, its messages are gone and it can't catch up (offset out of range). For strict broadcast guarantees to slow/offline subscribers, consider a compacted topic or a durable sink.
3. **Ordering is still per partition.** Every group sees the same per-partition order, but no cross-partition order — same rule as before. If a subscriber needs per-entity order, keep using keys (same key → same partition → ordered for every group).
4. **New subscribers start at `auto.offset.reset` .** A brand-new group either starts at `earliest` (replays history within retention) or `latest` (only new messages). Decide per subscriber. There's no "start from now" for an existing group without resetting offsets.
5. **Group coordination overhead per group.** Each group has a coordinator and rebalances. Many groups = more coordination traffic, but it's usually negligible unless you have thousands.
6. **No per-subscriber filtering at the broker.** Every group reads every partition; filtering is done client-side (or by using separate topics / a stream processor). If different subscribers need very different subsets, consider splitting topics or using Kafka Streams / a filter step.

Pattern summary

```
text
```

```
Producer ──► topic "events" (keyed by entity for ordering)

Subscriber 1:  group.id = "search-indexer"   subscribe("events")
Subscriber 2:  group.id = "audit-log"        subscribe("events")
Subscriber 3:  group.id = "notifier"         subscribe("events")
                 └─ 3 instances, same group.id "notifier" → they share the load,
                    but the notifier *group* still sees 100% of messages
```

Bottom line

> **Broadcast = distinct `group.id` per subscriber.** Same `group.id` = shared/queue semantics. You get fan-out for free from Kafka's group model — no special "broadcast topic" needed — but each subscriber pays the cost of a full independent read and manages its own lag/offsets against a shared retention window.

> Shortest answer, partitions = "what kind of things to receive", groups = "how many consumer units to broadcast to"

Not quite — that framing mixes them up. Shortest correct version:

- **Partitions** = **how the data is split** (units of ordering + parallelism). A record goes to exactly one partition.
- **Groups** = **who gets a copy** (units of broadcast). Each group receives *all* partitions; within a group, partitions are divided among its consumers.

So:

- **Partition** = *where* a message lives (one place).
- **Group** = *who* gets a copy (every group gets every message).
- **Consumer within a group** = *how many workers* share that group's copy.

Your sentence would be more accurate as:

> **Groups = "how many independent copies to broadcast." Consumers-per-group = "how many worker units split each copy." Partitions = "the split granularity."**

> Each group receives a copy of the event produced to the topic

Yes — **exactly right.** Every consumer group gets its own copy of every event produced

to the topic.

One precise caveat: the copy is **per partition, tracked by that group's own offsets** — so each group reads all partitions of the topic independently, from its own committed offsets. Same event, N groups = N independent reads/copies.

(Modulo the usual: `auto.offset.reset` for new groups, retention limits for far-behind groups, and possible redelivery during rebalances.)

> So the producer distributes the event to a given partition, every group receives it's copy of the event.

Yes — that's the correct model. One small wording fix:

- The **producer** doesn't "distribute to groups" — it writes the event to **one partition** of the topic (chosen by key/partitioner).
- **Every consumer group** then reads that partition independently and gets its **own copy** of the event (tracked by its own offset for that partition).

So the flow is:

```text
Producer ⟶ one partition of topic ⟶ every group reads it (own copy, own offset)
```

The partition decision happens **once, at produce time**, and is the same for all groups. Groups don't change *where* the event lands — they just each read *whatever* landed there.

> So keys could be invisible to the consumer, and multiple consumer machines can join the group and consume event copies in parallel?

Both statements are correct — with a small clarification on each.

Keys: yes, effectively "invisible" to the consumer

The key is **not part of the message payload** — it's metadata stored alongside the record (in the record batch). A consumer *can* access it if it wants:

```java
```

```
ConsumerRecord<String, Order> r = ...;
r.key();    // available if you choose to read it
r.value();  // the actual payload
```

But nothing forces a consumer to use it. Most consumers only care about the value. The key's job — **routing to a partition** — is already done by the time the consumer reads it. So from the consumer's perspective, the key is optional metadata it can ignore entirely. (It's "invisible" in the sense of not being embedded in your value/schema, not in the sense of being stripped.)

Multiple machines joining a group: yes, parallel consumption

Exactly. Consumers in the **same group** join and the coordinator **splits the partitions** among them:

```
text

Group "workers", topic with P0..P3

1 instance:   machine1 owns P0,P1,P2,P3
2 instances:  machine1 → P0,P1   machine2 → P2,P3
4 instances:  one partition each  (max parallelism)
5 instances:  one sits idle
```

Key properties:

- **Parallelism ceiling = partition count.** You can never have more *actively consuming* instances than partitions in the group.
- **Within the group, each partition is owned by exactly one instance** → no duplicate processing (except during rebalance redelivery).
- **Each instance gets a *slice* of the group's copy**, not the whole thing. The *group* still sees 100% of the topic.
- **Other groups are unaffected** — they run their own parallel consumer sets independently.
- **Scaling = add instances up to partition count**, then add partitions (note: changing partition count re-hashes keys).

Combined picture

```
text

Producer (key → partition P)

Topic: P0 P1 P2 P3
        |  |  |  |
Group A: m1 [P0,P1]   m2 [P2,P3]     ← parallel workers, one group copy
Group B: m3 [P0,P1,P2,P3]            ← another group, its own copy
Group C: m4 [P0] m5 [P1] m6 [P2] m7 [P3]  ← fully parallel, own copy
```

So: **keys route; groups fan out copies; instances within a group split the work in parallel.** The key is just routing metadata the consumer may ignore — it doesn't affect *how many* consumers can run in parallel (that's partitions), only *which* partition an event lands in.

> So in my broadcast case, every subscriber machine should have it's own group. but

...but then each subscriber machine gets **every** message, so you lose the ability to scale a subscriber horizontally. That's the tension. Let me name it precisely and give the options.

The tension

- **Broadcast (distinct group per machine):** every machine gets the full stream — but N machines = N× the read load, N× offsets, and no parallelism benefit. You're duplicating work, not sharing it.
- **Scale a subscriber (shared group):** machines split the partitions — but then the *group* sees everything and each *machine* sees only a slice.

You can't have both "every machine gets all messages" **and** "machines share the load" with the same group.id — they're opposite by design.

The key distinction: subscriber *service* vs. subscriber *machine*

This is usually where the confusion resolves:

> **One `group.id` per logical subscriber (service), NOT per machine.**

A logical subscriber = a service that needs the full stream. That service runs **N machines all sharing the same** `group.id` . Result:

- The **service** gets every message (it's one group). ✅ broadcast achieved.
- The **machines** split the partitions in parallel (same group). ✅ scales horizontally.
- **Different services** use different group.ids → each service gets its own full copy. ✅ fan-out.

```
text

Service "search-indexer" (group.id = "search-indexer")
   machine1 [P0,P1]   machine2 [P2,P3]   → service sees 100%, scales to 2

Service "audit-log"     (group.id = "audit-log")
   machine1 [P0..P3]                    → its own 100% copy
```

So the rule is:

> **Distinct** `group.id` **per logical subscriber/service. Same** `group.id` **across all machines of that service.**

When you'd genuinely want distinct groups per *machine*

Only if each machine is an **independent logical subscriber** that needs the whole stream and does *different* work — e.g.:

- Different tenants each needing the full feed, with per-tenant isolation.
- A "shadow" / canary consumer that mirrors production independently.
- Per-region or per-datacenter mirrors.

If they're doing the *same* work, that's just N duplicate workers — use one shared group instead.

Decision checklist

Your goal	group.id setup
Every service gets all events	**One group per service**
[illegible]	[illegible]

A service scales across machines	**All its machines share that one group.id**
Each machine independently needs the full stream	One group per machine (accept N× load)
Load-balance one job across machines	One group, all machines in it

The "but" you were about to say, restated

> "But if every subscriber machine has its own group, they each read everything and I can't parallelize within a subscriber."

Right — so **don't** give every *machine* its own group. Give every **service** its own group, and let that service's machines share it. That gives you broadcast-to-services *and* parallel-scaling-within-a-service simultaneously.

If your subscribers are truly independent single machines (no service-level grouping), then yes — one group each, and accept that horizontal scaling isn't a thing for them (or add partitions and more machines *within* that group if it ever needs to scale).

So partitions are for concurrent performance?

Yes — **partitions are primarily the unit of parallelism/concurrency**, which is what gives Kafka its throughput and horizontal scaling. But they're not *only* that; the same mechanism also gives ordering and compaction. Short version:

Partitions as a concurrency mechanism

- **Producer side:** more partitions = more parallel writes across brokers (a partition lives on one broker; different partitions spread across brokers/leaders). More partitions → more concurrent produce throughput.
- **Consumer side: max active consumers in a group = partition count.** This is the hard ceiling on consumer parallelism. More partitions → you can add more consumer machines to share the load.
- **Broker side:** partitions are the unit of replication and distribution across brokers, so they spread I/O and CPU.

So if someone asks "how do I scale Kafka consumption," the answer starts with "how many partitions do you have?" — that number caps your concurrency.

But partitions are also the unit of *ordering* and *state*

The same split that buys concurrency is what forces the "order only within a partition" rule, and it's the unit for:

- **Ordering** — guaranteed per partition, not per topic.
- **Offsets** — tracked per partition per group.
- **Compaction** — retention is per partition; compacted topics keep latest per key per partition.
- **Co-partitioning** — joins in Kafka Streams require matching partition counts.

So partitions are the **fundamental unit of parallelism *and* of ordering/state**. They're coupled: the reason you get concurrency is the reason you lose cross-partition order.

The trade-off (why not "just add partitions")

More partitions isn't free:

- **Rebalancing cost** grows with partition count.
- **Open file handles / memory** per partition on brokers and consumers.
- **End-to-end latency** can rise with many partitions (more batching/fetch overhead, longer rebalances).
- **Changing partition count re-hashes keys** → breaks per-key ordering/compaction placement mid-stream. Plan up front.
- **Diminishing returns** past a point — thousands of partitions per broker start to hurt.

Rule of thumb

> **Partitions ≈ the maximum concurrency you'll ever need, chosen up front.** Size it to your target consumer parallelism and throughput, add headroom, then leave it alone.

So: **yes, partitions are for concurrent performance** — that's their main practical purpose — but remember they simultaneously define ordering and state boundaries, which is why partition count is a design decision, not a tuning knob you spin freely.

> Is having dozens of keys (one per producer entity) a good idea in my system?

system.

Short answer: **yes, dozens of keys is totally fine and normal** — Kafka is designed for high-cardinality keys. The real question isn't "dozens?" (trivial) but **"is the distribution even, and does each key map to a partition you can live with?"** Let me give you the decision framework.

Dozens of keys is a non-issue

- Kafka handles **millions of distinct keys** routinely (user IDs, order IDs, device IDs). Dozens is nothing.
- Key cardinality doesn't cost anything extra at the broker — keys are just bytes in the record batch.
- More distinct keys actually **helps** distribution (see below).

So "is dozens of keys bad?" → **No. Never the problem.**

The real questions to ask

1. Are the keys evenly distributed in *volume*?

The partitioner is `hash(key) % numPartitions`. With dozens of keys and, say, 6 partitions, keys scatter across partitions — **but not evenly by traffic**. If one entity produces 90% of events, its partition is hot:

```
text

P0: keyA (90% of volume)   ← hot partition, bottleneck
P1: keyB, keyC, keyD
P2: keyE ... keyZ
```

Even key *count* ≠ even key *load*. Check your volume-per-key, not just key count.

2. Do you need per-key ordering?

If yes (almost always when keys = entities), **keep one key per entity** — same key → same partition → ordered. This is the main reason to key by entity at all. Dozens of keys is exactly the right shape for this.

3. Are dozens of keys enough to spread across partitions?

With `hash % N`, dozens of keys over a handful of partitions gives decent spread. But **if a single key dominates volume**, no amount of extra keys fixes the hot partition — you'd need to **salt** that key (see below).

4. Partition count vs. key count

- **Keys ≫ partitions:** fine, keys share partitions, each partition still ordered per key.
- **Keys ≈ partitions:** fine.
- **Keys < partitions:** some partitions may get no traffic for a given key set — uneven, but only matters if it starves consumer parallelism.

Rule: you want **enough keys (weighted by volume) to keep all partitions busy**, not necessarily keys == partitions.

When dozens of keys *is* a problem

- **Heavy skew:** one or two entities dominate → hot partition. Fix: **composite/salted key** (`entityId + "-" + bucket`) to split that entity across partitions — but note this **breaks per-entity ordering** unless you re-merge downstream. Only do it for keys that don't need strict order.
- **Tiny partition count:** dozens of keys over 2 partitions = poor parallelism. Add partitions (before you have data, ideally).
- **Key churn:** if "entity" keys are created/destroyed constantly (short-lived sessions), you still get ordering per key, but hot-key handling gets harder. Usually fine.
- **Compacted topic:** if you use `cleanup.policy=compact`, keys are the identity — dozens is fine, but every key retains its latest value forever (bounded by key count × value size). Dozens is negligible.

Practical guidance for your system

1. **Key by the entity you need ordered/co-located** (producer entity). Dozens of keys = correct.
2. **Measure volume per key.** If max-key volume ÷ average is large (say >5–10×), plan salting for that key or accept a hot partition.

3. **Size partitions to your target consumer parallelism**, not to key count. Dozens of keys over, e.g., 6–12 partitions is healthy.
4. **Don't add keys just to "spread load"** — adding low-volume keys doesn't fix a hot key; only salting/splitting the hot key does.
5. **Keep key stable** — don't change key format later (re-hashes → moves partitions → breaks ordering/compaction).

Bottom line

> **Dozens of keys is a good, normal design** — especially if keys = entities and you want per-entity ordering. The thing to watch is **volume skew per key**, not the key count. If one entity dominates, salt *that* key; otherwise leave it alone and size partitions for concurrency.

So: **yes, good idea** — just verify the hot-key distribution before you commit to the partition count.

> Can I passive poll consuming from multiple topics or do I have to spin poll?

You **must spin** `poll()` — there's no "passive" / push-based consumption in the standard Kafka client. The consumer is **pull-based**, and `poll()` is what drives *everything*: fetching, heartbeats, and rebalance participation. You can't sit idle and have messages arrive.

But you **can subscribe to multiple topics in one consumer** — that part is easy. Let me separate the two questions.

1. Multiple topics: yes, one `poll()` loop handles all of them

```java
consumer.subscribe(List.of("orders", "payments", "shipments"));

while (true) {
    ConsumerRecords<String, Event> records = consumer.poll(Duration.ofMillis(100));
    for (ConsumerRecord<String, Event> r : records) {
        // r.topic() tells you which one it came from
    }
}
```

- `subscribe()` takes a **list** of topics (or a regex).
- One `poll()` returns records **merged from all assigned partitions across all topics**.
- Use `record.topic()` (+ `partition()`, `offset()`) to route.
- Offset commits are per `TopicPartition`, so all topics are tracked independently under the hood.

So "consume from multiple topics" = **one loop, one consumer, multiple subscriptions**. No extra threads needed.

2. But it's still a spin loop — `poll()` is mandatory

There is no passive mode because:

- **Fetching is pull-based.** The broker doesn't push; the consumer asks. No `poll()` → no fetch.
- `poll()` **does heartbeats.** In the classic consumer, liveness (`session.timeout.ms`) is maintained by `poll()` calls. If you stop calling `poll()` longer than `max.poll.interval.ms`, the consumer is **kicked out of the group** and partitions rebalance. So even a slow/idle consumer must keep polling.
- `poll()` **drives rebalances.** Join/sync happens inside `poll()`.
- `poll()` **is also the commit/position machinery.** You advance positions by polling.

So the loop is the engine. "Passive poll" would mean an event-driven callback API — which the classic client **doesn't have** (there are wrappers/other clients, see below).

3. If you don't want a hand-rolled `while` loop

Options that give you a "just call me" feel:

Approach	What it does
Spring Kafka `@KafkaListener`	Framework runs the poll loop; your method is invoked per record/batch. You still *have* a spinning poller — just hidden.
Kafka Connect	Fully managed poll loop + framework; you write a connector, not a loop.

Reactor Kafka / reactive wrappers	Wraps `poll()` in a reactive stream. Still polling under the hood.
Vert.x Kafka client	Callback/event-loop style API over the same pull model.
`KafkaConsumer` **with** `wakeup()`	Lets you interrupt a blocking `poll()` from another thread — useful for graceful shutdown, not for passive mode.

All of these **still poll** — they just invert control so *you* don't write the loop. The pull model doesn't go away.

4. Common pitfalls with a multi-topic loop

- **Slow processing blocks all topics.** One `poll()` returns a batch; if you process it synchronously and slowly, you delay fetch for *every* topic/partition on that consumer. Keep per-batch work short or hand off to a worker pool (but then manage offsets carefully — commit only after processing).
- `max.poll.records` **and** `max.poll.interval.ms` bound how much you can fetch and how long you may take between polls. Tune for your slowest batch.
- **Skewed topics.** One high-volume topic can starve the others in a single-threaded loop. If topics have very different throughput/processing needs, consider **separate consumers (threads/instances) per topic** — each still spinning its own `poll()`.
- **Regex subscription** re-evaluates on metadata refresh; new matching topics get picked up automatically (nice for dynamic fan-in).
- **Error isolation.** A poison record in topic A shouldn't kill the loop for topic B — handle per-record errors and decide on skip/DLQ.

Bottom line

> **Multiple topics: yes — one** `subscribe(List.of(...))` **and one** `poll()` **loop covers all of them. Passive: no — you must keep calling** `poll()`; it's the pull engine *and* the heartbeat/rebalance driver. If you want callback-style code, use a framework (Spring Kafka, Connect, reactive wrapper) — but underneath, something is still spinning `poll()`.

So: **spin the poll loop (or let a framework spin it for you), and subscribe to as many topics as you want in that one loop.**

> Shortest answer, advice, should topics be bidirectional or monodirectional?

Monodirectional. One topic = one direction/stream. Don't mix requests and replies (or reads and writes) in the same topic.

- Requests and replies → **separate topics** (`orders.requests` , `orders.replies`).
- Different event types with different ordering/retention/consumer needs → **separate topics**.
- Bidirectional in one topic breaks ordering assumptions, complicates compaction, and forces every consumer to filter out half the messages.

Rule: **topics are streams, not channels.** If two flows need different schemas, keys, retention, or consumers, they're two topics.

> in python confluent_kafka can I see the topic from where I received the data?

Yes. The `Message` object gives you `.topic()` (and `.partition()` , `.offset()` , `.key()` , `.value()` , `.headers()` , `.timestamp()`).

```python
from confluent_kafka import Consumer

c = Consumer({"bootstrap.servers": "localhost:9092",
              "group.id": "g1",
              "auto.offset.reset": "earliest"})
c.subscribe(["orders", "payments"])

while True:
    msg = c.poll(1.0)
    if msg is None:
        continue
    if msg.error():
        print("err:", msg.error())
        continue

    print(msg.topic(), msg.partition(), msg.offset(), msg.key(), msg.value())
```

So for routing a merged multi-topic stream: `msg.topic()` is what you check.

Quick notes:

- Works the same whether you `subscribe([...])` or `assign([...])`.
- `msg.topic()` returns a `str`; `msg.key()` / `msg.value()` are `bytes` (decode yourself, e.g. `.decode("utf-8")`).
- If you want to route by topic, build a dict like `handlers = {"orders": handle_orders, "payments": handle_payments}` and call `handlers[msg.topic()](msg)`.
- `consumer.committed(...)` / `consumer.position(...)` also take `TopicPartition` if you need to inspect offsets per topic.